

Projet Informatique de 2ème année

Groupe 32

EX-Libris

Étudiants :

Arnaud BICHON
Axel LECLERCQ
Cécile LOCATELLI
Moussa SOULAMA
Anne-Camille TAMPÉ
Paul FOURCADE

Encadrant :

Elwenn JOUBREL

Table des matières

1	Introduction	3
2	Cahier des Charges	4
2.1	Présentation d'Open Library	4
2.2	Diagramme de cas d'utilisation	6
2.3	Diagramme d'activité	7
3	Organisation du projet	8
3.1	Organisation du groupe	8
3.2	Diagramme de Gantt	9
4	Maquette de l'application	10
4.1	Diagrammes BDD	10
4.2	Diagramme de classes	10
4.2.1	Classes Business Object	10
4.2.2	Classes DAO	11
4.2.3	Classes Service	11
4.3	Diagrammes de séquence	12

1 Introduction

Depuis plusieurs années, le numérique a profondément transformé la manière dont nous consommons et partageons nos loisirs culturels. Le cinéma dispose de Letterboxd, la musique de Last.fm : des plateformes communautaires qui permettent à chacun de garder une trace de ses découvertes, de les noter et d'échanger avec d'autres passionnés. La lecture, pourtant, reste étonnamment mal servie sur ce plan. Le principal service existant, Goodreads, appartient à Amazon, ce qui pose des questions en matière d'indépendance éditoriale et d'exploitation des données personnelles des lecteurs.

C'est de ce constat qu'est né le projet Ex-Libris : une application de suivi de lecture pensée comme un véritable réseau social pour les lecteurs. L'objectif n'est pas de reconstruire un catalogue de livres, un travail déjà accompli et maintenu par la plateforme collaborative Open Library, qui référence plusieurs millions d'ouvrages en accès libre. Ce qu'Open Library ne propose pas, en revanche, c'est la dimension humaine de la lecture : qui a lu quel livre, quand, avec quel avis, et quelles recommandations peuvent en être tirées. C'est précisément cette couche que notre application vient ajouter.

Concrètement, Ex-Libris doit permettre à un utilisateur de créer un compte, de rechercher un livre et d'en consulter la fiche détaillée, de l'ajouter à sa bibliothèque personnelle avec un statut de lecture, de le noter et d'en rédiger une critique. Au-delà de cette gestion individuelle, l'application intègre une dimension sociale : chaque utilisateur peut consulter le profil d'un autre lecteur, s'abonner à son activité, et bénéficier d'un système de recommandation construit à partir des affinités entre utilisateurs.

D'un point de vue technique, le projet repose sur une API REST développée avec FastAPI, exploitant les données de l'API publique Open Library et s'appuyant sur une base de données PostgreSQL pour la persistance des données propres à notre application. Le langage retenu est Python, dans une logique de programmation orientée objet.

Ce dossier présente dans un premier temps le cahier des charges du projet, les choix et arbitrages qui ont guidé nos réflexions, ainsi que l'organisation retenue au sein de notre équipe. Il détaille ensuite le modèle de conception élaboré pour répondre à ces besoins, à travers les différents diagrammes UML réalisés.

2 Cahier des Charges

Ce cahier des charges exprime le besoin initial auquel notre application doit répondre : une application de suivi de lecture combinant la richesse du catalogue d'Open Library à une véritable dimension sociale entre lecteurs. Sur cette base, notre travail a consisté à reformuler ce besoin, à en préciser le périmètre exact, et à définir les fonctionnalités attendues ainsi que les contraintes techniques qui en découlent, notamment celles liées à l'exploitation d'une source de données externe.

2.1 Présentation d'Open Library



Notre application repose entièrement sur les données fournies par Open Library, une base collaborative et gratuite qui référence plusieurs millions d'ouvrages. Il s'agit d'une API web : on l'interroge en construisant des adresses (URLs) spécifiques, et elle nous renvoie les données demandées au format JSON.

Besoin	URL type	Exemple
Rechercher des livres	<code>search.json?q=...</code>	<code>openlibrary.org/search.json?q=le+petit+prince</code>
Voir une œuvre précise	<code>/works/{id}.json</code>	<code>openlibrary.org/works/OL10263W.json</code>
Voir une édition précise	<code>/books/{id}.json</code>	<code>openlibrary.org/books/OL7366018M.json</code>
Voir un auteur précis	<code>/authors/{id}.json</code>	<code>openlibrary.org/authors/OL31901A.json</code>
Voir la définition officielle d'un type	<code>/type/{nom}</code>	<code>openlibrary.org/type/work</code>

Pour bien comprendre comment exploiter ces données, il est essentiel de saisir la façon dont Open Library organise ses informations. La base distingue en réalité trois notions différentes qui, dans le langage courant, sont toutes désignées par le mot “livre” :

- L'auteur (Author) : la personne ayant écrit l'œuvre.
- L'œuvre (Work) : il s'agit de l'idée générale du livre, indépendamment de toute édition particulière. C'est à ce niveau que l'on trouve par exemple le titre, un résumé, ou les grands thèmes abordés.
- L'édition (Edition) : il s'agit d'une publication précise et matérielle de cette œuvre, par exemple une édition Harcourt de 1971. Une même œuvre peut avoir des centaines d'éditions différentes (traductions, rééditions, formats poche...).

Dans notre base de données, nous avons fait le choix de conserver, pour chaque œuvre, l'ensemble de ses éditions, afin d'offrir aux utilisateurs une réelle variété de choix (langue, édition, format) lorsqu'ils ajoutent un livre à leur bibliothèque. Nous détaillerons la façon

dont nous avons stocké et traité ces données dans la suite de ce dossier d'analyse, lors de la conception de la maquette.

Nous avons également constaté que certaines données présentes dans Open Library manquent de cohérence, notamment les thèmes et genres associés aux œuvres (champ subjects), qui sont rédigés en texte libre et mélangent parfois de véritables genres littéraires avec de simples mots-clés. Cette hétérogénéité a directement motivé l'une de nos réflexions pour la fonctionnalité de normalisation des genres, qui vise à proposer une classification plus cohérente et plus facilement exploitable pour la recherche par genre.

Type Work

Champ	Description
title	Titre de l'œuvre
subtitle	Sous-titre
authors[]	Auteur(s) + leur rôle (via author_role)
subjects[]	Thèmes/genres (peu normalisés)
subject_places[]	Lieux liés à l'œuvre
subject_times[]	Époques liées à l'œuvre
subject_people[]	Personnes liées à l'œuvre
description	Résumé/description
dewey_number[]	Classification Dewey (bibliothéconomie)
first_sentence	Première phrase de l'œuvre
original_languages[]	Langue(s) d'origine
first_publish_date	Date de toute première publication
cover_edition	Édition choisie pour la couverture affichée
covers[]	Identifiants d'images de couverture
genres[]	Tags de classification (système différent de subjects)
identifiers	Liens vers bases externes (Wikidata, Goodreads...)

Type Edition

Champ	Description
title	Titre de cette édition
authors[]	Auteur(s) (référence directe, pas de rôle ici)
isbn_10 / isbn_13	Identifiant international, propre à cette édition
number_of_pages	Nombre de pages de cette édition précise
publishers[]	Éditeur(s)
publish_date	Date de CETTE publication
physical_format	Format (poche, relié, cartonné...)
languages[]	Langue de cette édition/traduction
works[]	Lien retour vers le Work parent
subjects[] / genres[]	Existent officiellement mais rarement remplis à ce niveau
series[]	Appartenance à une saga/collection
translation_of	Indique si c'est une traduction
source_records[]	Traçabilité des sources ayant alimenté la fiche

Type Author

Champ	Description
name / personal_name	Nom de l'auteur
title	Attention : ici ce n'est PAS un titre de livre, mais le nom sous forme "Nom, Prénom" (tri alphabétique)
alternate_names[]	Variante/orthographe du nom rencontrées dans les sources
bio	Biographie
birth_date / death_date	Dates de naissance/mort
photos[]	Identifiants de photos
remote_ids	Liens vers bases externes (VIAF, Wikidata, ISNI)
location	Lieu associé à l'auteur
wikipedia	Lien Wikipedia

2.2 Diagramme de cas d'utilisation

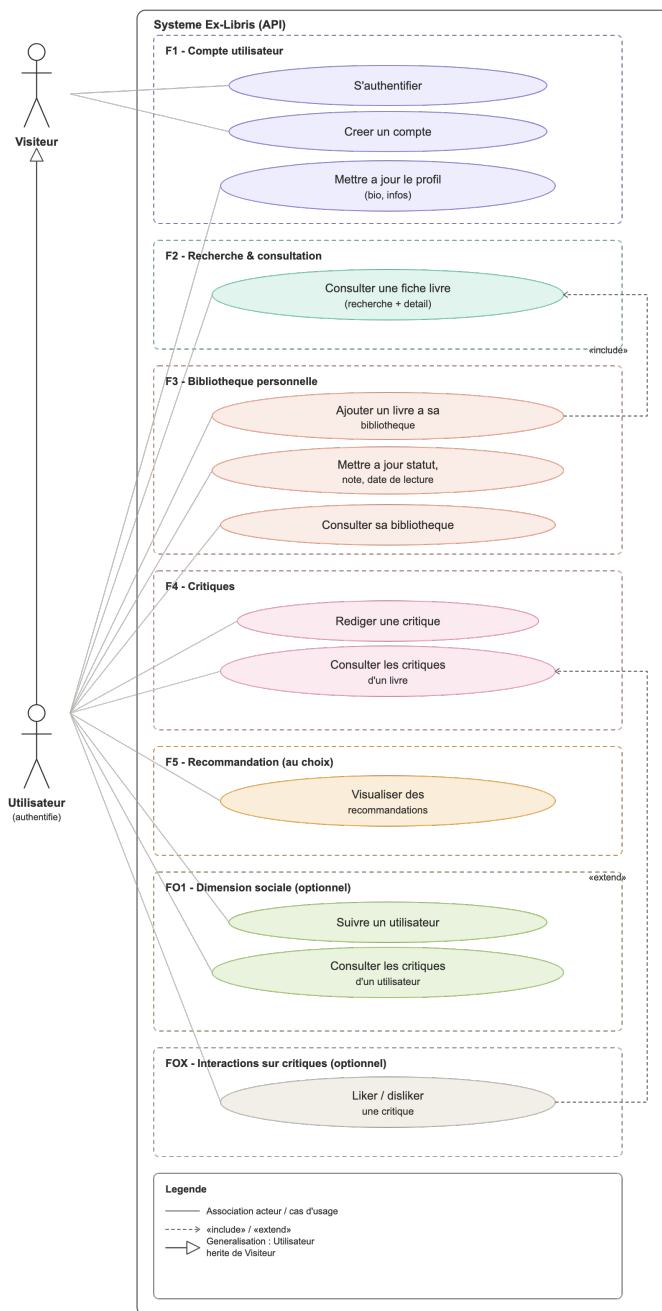


Figure 11 – Diagramme de cas d'utilisation

2.3 Diagramme d'activité

3 Organisation du projet

3.1 Organisation du groupe

Dès le début du projet, nous avons désigné Anne-Camille Tampet comme cheffe de projet et Cécile Locatelli comme leader technique, en veillant, conformément aux conseils donnés en début de projet à ne pas cumuler ces deux rôles sur une même personne.

Pour communiquer, nous avons créé un groupe sur Teams, sur lequel chacun peut partager ses avancées, poser une question ou signaler un point de blocage dès que le besoin s'en fait sentir. C'est notre principal canal d'échange, sur lequel nous discutons régulièrement. En parallèle, nous avons mis en place un dépôt partagé sur GitHub, accessible à tous les membres du groupe, qui nous sert à centraliser et à distribuer l'ensemble des fichiers importants du projet.

Concernant la répartition du travail, notre organisation a surtout été pensée pour cette première phase d'analyse. Chaque membre du groupe s'est vu confier la réalisation d'un diagramme ou d'une partie spécifique du dossier, de façon à ce que chacun puisse s'investir concrètement dans la réflexion et la conception du projet. Cette répartition individuelle ne nous a cependant pas empêchés de travailler de façon très collective. Nous avons pris l'habitude de nous retrouver au minimum deux fois par semaine pour faire le point ensemble. Ces moments nous permettaient de relire et de discuter les diagrammes réalisés par chacun, de nous assurer que tout le monde comprenait bien les choix faits par les autres et de les faire évoluer si nécessaire. Cette phase de discussion collective a été particulièrement importante en tout début de projet. Elle nous a permis de poser une base commune solide et de nous assurer que tout le monde partageait la même vision de l'application avant d'avancer plus en détail sur la partie codage.

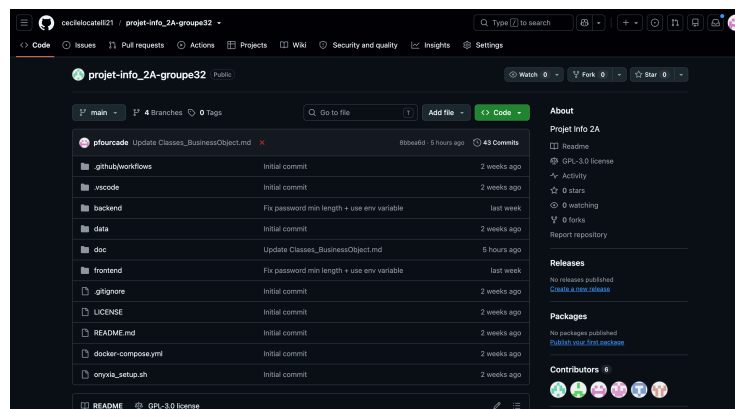


Figure 11 – Dépôt Github

3.2 Diagramme de Gantt

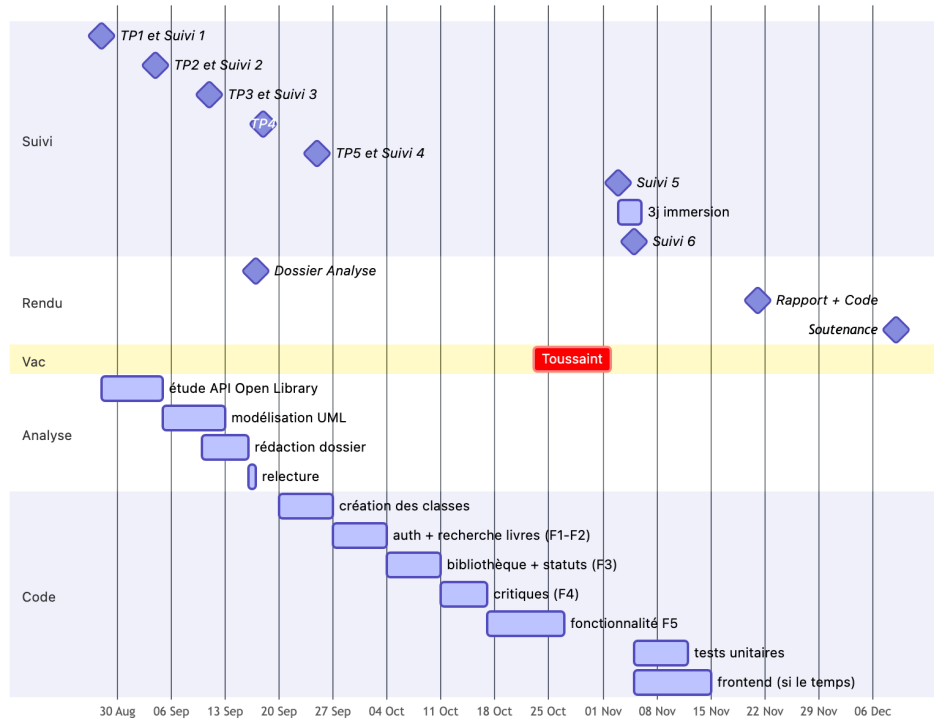


Figure 11 – Diagramme de Gantt

Notre planning se découpe en deux grandes phases. D'abord la phase d'analyse, de fin août à fin septembre : nous commençons par étudier l'API Open Library, puis nous réalisons l'ensemble de la modélisation UML afin de bien comprendre comment notre application va s'articuler et comment la construire.

Une fois cette base posée, nous passons à la phase de code, d'octobre à début novembre. Nous avons choisi de développer les fonctionnalités par ordre de difficulté croissante : d'abord les classes de base, puis l'authentification et la recherche de livres (F1-F2), la bibliothèque avec les statuts de lecture (F3), les critiques (F4), et enfin notre fonctionnalité F5 (les recommandations), qui nécessitera davantage de temps du fait de sa complexité. Les tests unitaires et le frontend interviendront en dernier, le frontend n'étant pas prioritaire, sera développé uniquement s'il nous reste du temps.

4 Maquette de l'application

4.1 Diagrammes BDD

4.2 Diagramme de classes

4.2.1 Classes Business Object

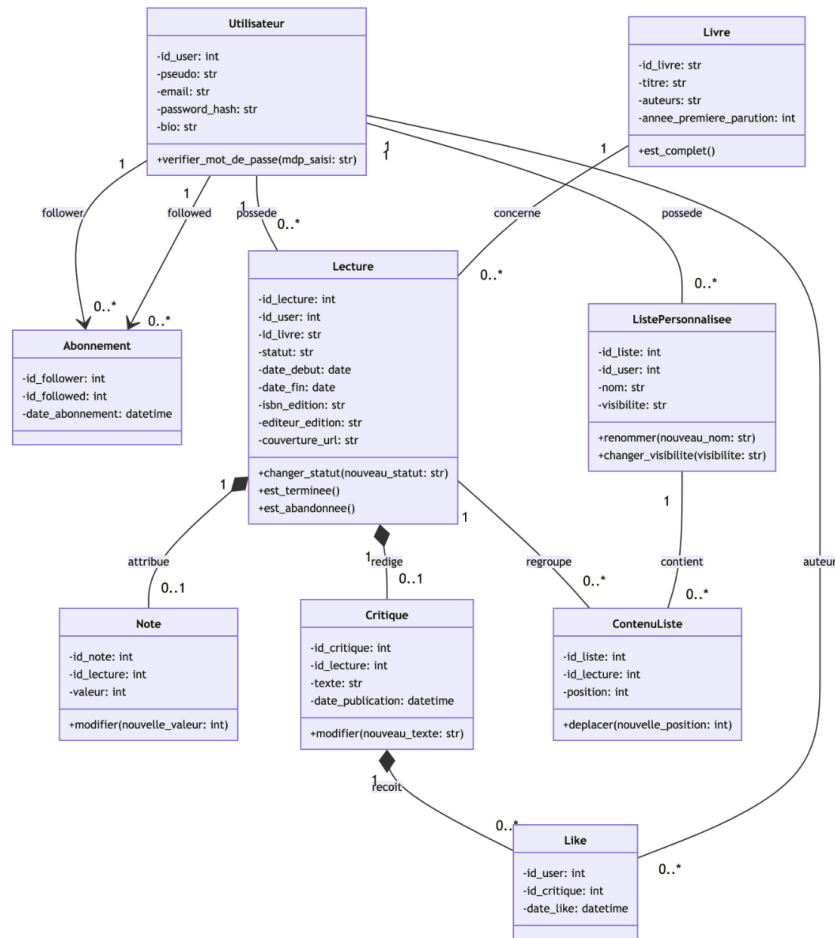


Figure 11 – Diagramme de classe : Business Object

4.2.2 Classes DAO

4.2.3 Classes Service

4.3 Diagrammes de séquence

Les diagrammes de séquence suivant permettent de détailler la chronologie des échanges entre les différents composants de notre architecture (Utilisateur, Front End, Back End, DAO, API externe) pour trois fonctionnalités de l'application : la suppression d'une fiche de lecture, la sélection d'un livre et la demande de recommandations personnalisées.

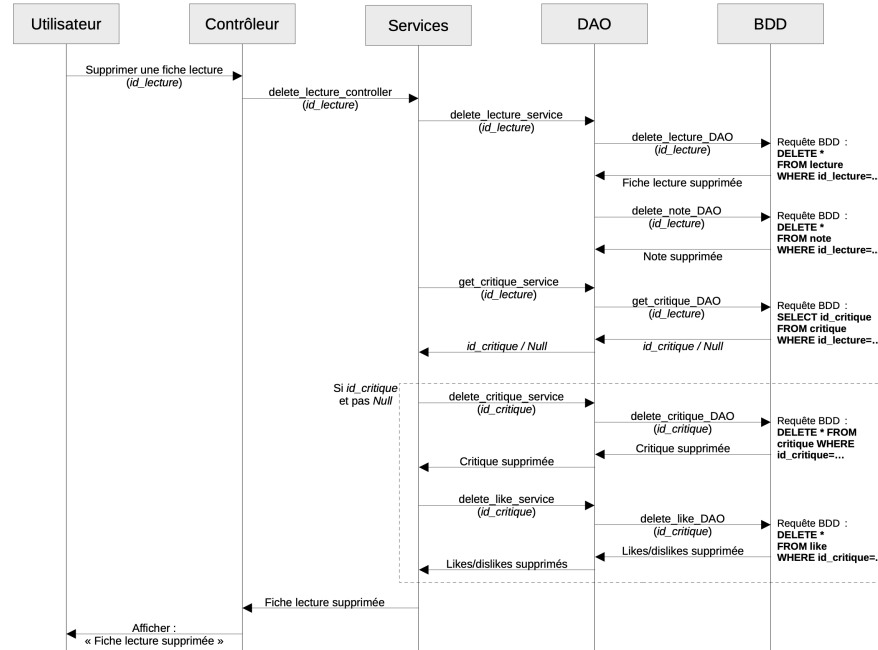


Figure 11 – Diagramme de séquence : supprimer une fiche de lecture

Le but de cette séquence est de supprimer une fiche lecture pour un utilisateur.

Celui-ci a créé auparavant une fiche lecture : il a enregistré un livre avec des informations (statut du livre, note, etc., il a éventuellement ajouté une critique associée). Il souhaite aujourd'hui la supprimer.

Il se situe au départ sur la fiche lecture considérée, et clique sur un bouton « Supprimer la fiche lecture ».

L'information, avec l'*id_lecture* (l'identifiant de la fiche lecture) est réceptionnée par le **Contrôleur**, qui la transmet au **Service**, qui envoie des requêtes au **DAO**.

Dans un premier temps, la fiche lecture est supprimée de la table de données « **lecture** », ainsi que sa *note* associée.

Ensuite, le **Service** va rechercher dans la base de données l'existence éventuelle d'une *critique* associée à cette lecture. Si une *critique* est trouvée, alors celle-ci sera supprimée, ainsi que d'éventuels likes/dislikes associés.

À la fin de toutes ces opérations, le **Service** transmet au **Contrôleur** l'information que la fiche a bien été supprimée, celui-ci affiche à l'écran de l'utilisateur « La fiche lecture a bien été supprimée ».

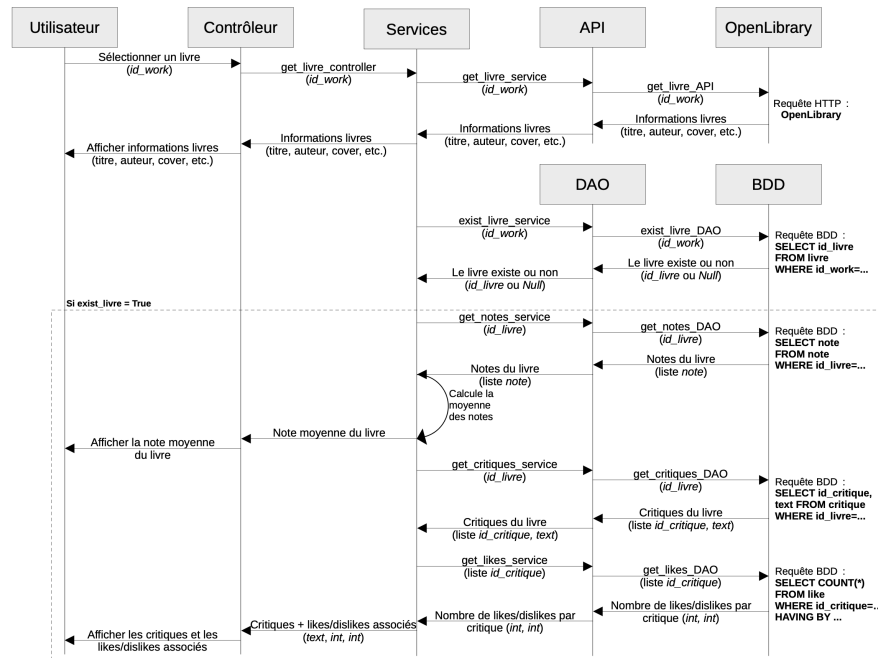


Figure 11 – Diagramme de séquence : sélectionner un livre

Le but de cette séquence est d'afficher toutes les informations disponibles sur un livre sélectionné par un utilisateur.

Celui-ci a par exemple effectué une recherche, a obtenu une liste de livres (correspondant à une liste de *id_work*, les identifiants stockés par **OpenLibrary**), il choisit l'un d'entre eux pour en afficher toutes les informations.

La première étape est de récupérer les informations fournies par **OpenLibrary** sur ce livre : le **Contrôleur** va transmettre au **Service** l'*id_work* considéré, pour utiliser l'API qui va effectuer une requête au niveau d'Open Library.

De cette requête, on va récupérer les informations pertinentes contenues dans **OpenLibrary** (titre, auteur, date, cover, etc. – la liste précise reste à définir).

Ensuite, il s'agit de fournir les informations propres à notre application Ex-Libris.

Pour cela, le **Service** va transmettre l'*id_work* au **DAO**, qui va rechercher dans la base de données si un livre a déjà été enregistré associé à celui-ci. Si oui, le **DAO** remontera au **Service** un *id_livre* correspondant.

À partir de cet *id_livre*, le **Service** va demander au **DAO** de récupérer toutes les notes (*note*) mises par les utilisateurs à ce livre, pour ensuite en faire la moyenne.

Il va ensuite rechercher les critiques (critique) associées à ce livre, et les likes/dislikes (*like*) associés. Si de nombreuses critiques ont été rédigées sur ce livre, il sera possible de n'en choisir que quelques-unes (par exemple, celles avec le plus grand nombre de likes), même si cela n'apparaît pas dans le diagramme pour éviter une surcharge visuelle.

Le **Service** transmet donc au **Contrôleur** les informations suivantes : celles issues d'**OpenLibrary** d'abord ; et si le livre existe déjà dans Ex-Libris, il ajoutera la note moyenne, les critiques et les likes/dislikes associés à ces critiques.

Enfin, le **Contrôleur** mettra en forme toutes ces informations pour les afficher sur la page utilisateur.

À noter que dans cette séquence, si le livre n'existe pas dans la base de données Ex-Libris, il n'est pas ajouté à cette base. En effet, il a été convenu par l'équipe projet qu'un ajout ne se fait qu'au moment où un utilisateur enregistre le livre dans sa bibliothèque.

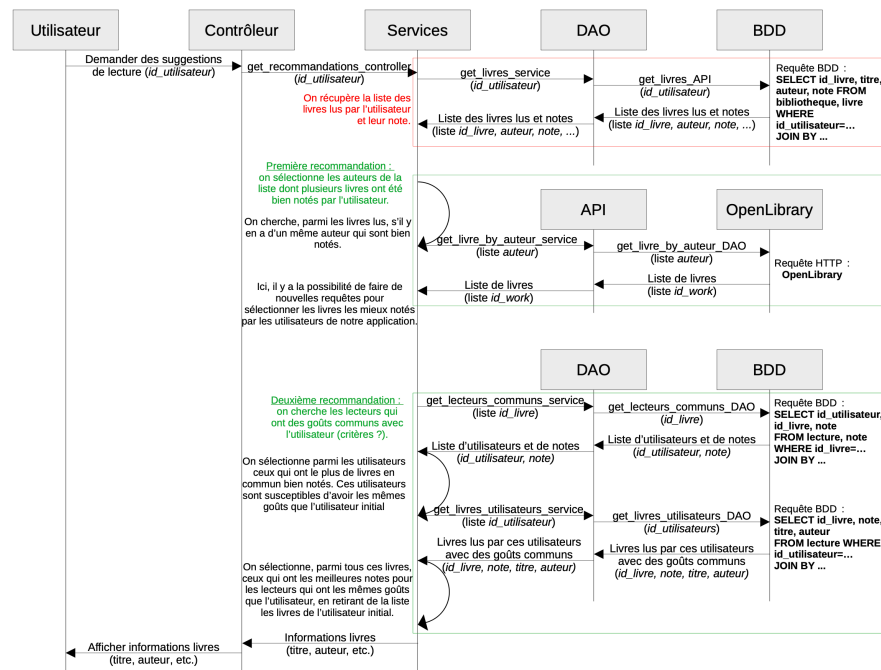


Figure 11 – Diagramme de séquence : sélectionner un livre

Le but de cette séquence est de recommander à l'utilisateur des livres qu'il n'a pas enregistrés et qui seraient susceptibles de lui plaire.

Il se situe sur sa page d'accueil, et clique sur un bouton « Recommandations personnelles ».

La première étape est de récupérer la liste des livres lus par cet utilisateur (*id_livre*), ainsi que les notes associées à chacun de ces livres (*note*). On se sert pour cela de la table 'bibliotheque', qui enregistre les lectures d'un utilisateur.

Nous envisageons ensuite plusieurs possibilités pour effectuer des recommandations.

Premier type de recommandations :

Si dans les livres lus par l'utilisateur, plusieurs livres d'un même auteur sont bien notés, d'autres livres du même auteur lui seront proposés.

Le **Service** va donc essayer d'identifier dans la liste de livre les auteurs les plus appréciés. S'il trouve des candidats, il va interroger **OpenLibrary**, via l'**API**, pour rechercher d'autres livres des mêmes auteurs.

Il est également possible de rechercher, parmi ces livres d'auteurs appréciés par l'utilisateur, ceux qui ont les meilleures notes dans notre application Ex-Libris, mais cela n'apparaît que comme commentaire sur le diagramme de séquence pour ne pas alourdir le schéma.

Deuxième type de recommandations :

L'objectif ici est d'identifier dans Ex-Libris d'autres utilisateurs ayant des goûts similaires à ceux de l'utilisateur initial.

Pour cela, le **Service** va récupérer dans la base de donnée, via le **DAO**, la liste de tous les utilisateurs ayant des livres en commun avec l'utilisateur initial, ainsi que les notes associées.

Cette requête pouvant être potentiellement lourde en calculs et en stockage mémoire, il est envisageable de l'affiner pour ne choisir que les livres les mieux notés par exemple (des notes de 5/5, puis si cela ne suffit pas, considérer les livres avec des notes de 4/5, etc.)

Une autre possibilité, si l'application rencontre un immense succès et qu'un très grand nombre d'utilisateurs y rentrent des dizaines de livres, serait de modifier la structure de la base de donnée afin de faire apparaître une table de relations entre les utilisateurs, qui serait mise à jour à chaque note mise sur un livre par exemple.

Mais, dans un premier temps, nous allons considérer que la simple requête brute initialement décrite doit fonctionner correctement avec quelques milliers d'utilisateurs.

Ainsi, nous disposons d'une liste d'utilisateurs (*id_utilisateur*) que l'on va considérer avoir des goûts relativement proches de ceux de l'utilisateur initial.

Le **Service** va alors rechercher dans la base de données, via le **DAO**, tous les autres livres appréciés par ces utilisateurs (récupération de tous les livres, classement par notes, élimination des livres dans la bibliothèque de l'utilisateur).

Nous obtenons donc une liste de livres, dont l'auteur est apprécié par l'utilisateur, ou qui ont été appréciés par des lecteurs aux goûts relativement proche de ceux de l'utilisateur.

Le **Service** renvoie alors ces informations au **Contrôleur**, qui les affiche sur l'écran de l'utilisateur.

D'autres algorithmes de recommandations pourront être envisagés dans un second temps, une fois l'architecture de l'application suffisamment développée.

partie 1 : Cahier des Charges partie 2 : Maquette de l'application

Suivi projet Ex-Libris

Fait

- Arno : diagramme de séquence (partie 2)
- Paul : diagramme de Gantt (partie 1) paul le caca qui pue berkklkk
- Cecile : Diagramme de cas d'utilisation (partie 1)

En cours

- Moussa : diagramme BDD (partie 2)
- Axel : diagramme d'activité (partie 1)
- Anne-Camille : diagramme de classe business (détail + explication des classes) (partie 2)
- Paul : Introduction

À faire

- Partie répartition des tâches et présentation de l'équipe (partie 2)
- Diagramme de package (DAO / Services / Business / View) (partie 2)
- Détail d'un service (partie 2)
- Présentation de l'API (partie 1) ...